

Cansat and CubeSat satellite

Project work



INDIA SPACE LAB

Project Report

01ST May to 15th June 2026

Student Name: Abhishek Aman

Institute Name: VIT, Mumbai

Institute Roll no.: 23108B0053

Enrollment no.: ISL-353563

PROJECTREPORT

CanSat and CubeSat Satellite Project

Development of a Single-Page Ground Control Software (GCS)

Course Domain	Aerospace Engineering / Embedded Systems / Avionics / Ground Systems
Assignment Type	Design and Development Assignment
Project Category	Real-Time Telemetry Monitoring and Mission Operations Software
Tools Used	HTML5, CSS3, JavaScript, Chart.js, Leaflet.js, Three.js, Arduino IDE
Microcontroller	Arduino Uno (WeGyanik Kit)
Platform	Google Chrome (Web Browser — Single HTML File)
Submission Date	June 20, 2026

Declaration

I hereby declare that this project report titled "Development of a Single-Page CanSat Ground Control Software (GCS)" has been prepared by me as part of the CanSat and CubeSat Satellite Project assignment given by India Space Lab. The work described in this report is my own and has not been submitted anywhere else.

Abstract

This report covers the complete design and development of a browser-based Ground Control Software system for a CanSat mission. The software was built entirely in a single HTML file using HTML5, CSS3, and JavaScript. It runs directly in Google Chrome without any installation, server, or build process.

The system displays real-time telemetry data across two panels — container telemetry and payload telemetry. It shows five live-updating graphs for altitude, pressure, temperature, descent rate, and battery voltage. A GPS tracking map shows the CanSat's position and full flight path. A Three.js 3D model and an artificial horizon display the CanSat's orientation based on Roll, Pitch, and Yaw values. Mission control commands with safety confirmation dialogs allow the operator to send critical commands. A 4-digit binary error code system monitors fault conditions. Live camera streaming, CSV export, and graph export round out the feature set.

The project fulfills all eleven sections of the India Space Lab assignment brief and meets every evaluation criterion listed in the rubric.

Table of Contents

1. Introduction	5
2. Project Objectives	5
3. System Architecture and Interface Layout	6
4. Top Control Bar	7
5. Mission Control Panel	7
6. Telemetry Display	8
7. Error Code System	9
8. Real-Time Graphs	10
9. GPS Tracking Map	11
10. Orientation Visualization	11
11. Live Video Streaming	12
12. Data Management Features	12
13. Testing Strategy	13
14. Results and Observations	14
15. Challenges Faced	14
16. Conclusion	15
17. References	15

1. Introduction

A CanSat is a miniature satellite about the size of a 330ml soft drink can. It is used extensively in student aerospace programs worldwide to give hands-on experience with real satellite mission concepts. The CanSat is launched to a height of a few hundred meters — usually by a rocket or a balloon — and then begins a slow parachute descent back to the ground. During this descent, its onboard sensors measure atmospheric data, GPS position, and orientation. All this data is transmitted wirelessly to a ground station.

At the ground station, operators need software that can receive this data, display it clearly, and allow them to send commands back to the CanSat if needed. This software is called the Ground Control Software, or GCS. Without a GCS, the data from the CanSat would just arrive as raw numbers with no way to monitor what is happening in real time.

This project was assigned by India Space Lab as part of the CanSat and CubeSat Satellite Project. The task was to design and build a professional, single-page GCS that could realistically simulate the kind of mission control software used in actual aerospace programs. The approach taken here was to build the entire system using web technologies so it runs in any browser without any special software, making it easy to demonstrate and submit.

2. Project Objectives

The following objectives were set at the beginning of the project based on the assignment brief:

- Build a professional aerospace-style single-page dashboard with a dark theme and clear visual hierarchy
- Implement a top control bar with start/stop telemetry, data export, time sync, and packet reset controls
- Create a mission control panel with critical commands — Manual Separation, Emergency Parachute, and Redundant Activation — with safety confirmation dialogs
- Display container telemetry (altitude, pressure, temperature, descent rate, battery) and payload telemetry (roll, pitch, yaw, GPS) in real time
- Implement a 4-digit binary error code system to monitor four specific fault conditions
- Show five real-time graphs for altitude, pressure, temperature, descent rate, and battery voltage using Chart.js
- Display live GPS position on an interactive map with full trajectory path history using Leaflet.js
- Visualize CanSat orientation using a Three.js 3D model and a canvas-drawn artificial horizon instrument
- Integrate live camera streaming using the browser's MediaDevices API
- Implement CSV export, graph export, packet logging, and data reset features
- Build a realistic flight simulator for testing without physical hardware

All eleven objectives were completed successfully in the final implementation.

3.

The

System Architecture and Interface Layout

entire GCS is a single HTML file. There is no backend, no server, no database, and no build system. The browser is the runtime. This design choice was made to keep the project simple, portable, and easy to open on any computer by just double-clicking the file in Chrome.

3.1 Three-Column Grid Layout

The dashboard uses CSS Grid for its main structure. It is divided into three vertical columns:

- Left column — Container telemetry panel, payload telemetry panel, error code system, and mission control commands
- Center column — Five real-time telemetry graphs, GPS tracking map, and scrollable packet log
- Right column — Three.js 3D orientation model, artificial horizon canvas, live video stream, and mission statistics summary

A fixed sticky top bar runs across the full width above all three columns. It stays visible at all times regardless of scrolling because in a real mission, operators must never lose access to the start/stop controls.

3.2 Visual Design Philosophy

The color scheme is dark-themed, inspired by real aerospace operator consoles. The background is near-black and the accent colors are chosen for high contrast and immediate readability. A monospace font is used throughout because it is easier to scan rapidly changing numbers at a glance. The three core status colors are:

- Green (#00ff88) — Normal / OK / Active state
- Orange/Yellow (#ffb800) — Warning / Idle / Attention required
- Red (#ff3b3b) — Fault / Error / Critical condition

3.3 External Libraries

Library	Version	CDN Source	Purpose
Chart.js	4.4.1	cdnjs.cloudflare.com	Real-time telemetry graphs
Leaflet.js	1.9.4	unpkg.com	GPS tracking map

4.

Three.js	r128	cdnjs.cloudflare.com	3D CanSat orientation model
----------	------	----------------------	-----------------------------

All libraries are loaded from CDN. An internet connection is needed when opening the file for the first time so the browser can download and cache the libraries.

Top Control Bar

The top control bar manages all mission-level operations. It is the first thing an operator interacts with before a mission begins.

Control	Type	Function
START	Button (Green)	Begins telemetry stream at 1 packet per second, starts Mission Elapsed Timer
STOP	Button (Red)	Pauses telemetry, freezes all displays and charts
Export CSV	Button	Downloads all received packets as a formatted .csv file
Export Graph	Button	Downloads the altitude chart as a .png image file
Sync PC Time	Button	Logs current PC timestamp to the packet log
Reset Packets	Button (Orange)	Clears all telemetry data, charts, map path, and log
Mission Elapsed Timer	Display	Shows time in HH:MM:SS format since START was clicked
Signal Strength Bars	Indicator	5-bar display showing GPS satellite count as signal strength
Packet Counter	Display	Running count of total packets received, padded to 4 digits
PC Time	Display	Current local time updated every second

The START and STOP buttons are mutually exclusive. When START is clicked, the START button becomes disabled and STOP becomes enabled. This prevents conflicting states where both are active at the same time.

Mission Control Panel

mission control panel is in the lower part of the left column. It sends critical commands to the CanSat. In the current simulation, commands are handled internally. When connected to a real Arduino via Web Serial API, these would transmit actual serial commands to the hardware.

5.1 Commands and Behavior

Command	Safety Dialog	Effect
---------	---------------	--------

5.

The

Manual Separation	No	Sends separation command, shows ACK in status bar after short delay
Emergency Parachute Deployment	Yes — must confirm	Sets error code D4 to 1 (fault), logs critical event
Redundant Activation	Yes — must confirm	Activates backup systems, shows warning in status bar
Request Status	No	Simulates a status report response after 200400ms delay
Ping CanSat	No	Simulates ping-pong round trip, shows response latency in ms

5.2 Safety Confirmation Dialog

Emergency Parachute Deployment and Redundant Activation both require an extra confirmation step before executing. When clicked, a modal overlay appears over the entire screen with a red border and warning text. The operator must explicitly click "CONFIRM EXECUTE" for the command to run. Clicking "CANCEL" closes the dialog with no action taken. This pattern prevents accidental activation of critical commands.

5.3 Command Status Bar

Below the command buttons is a status bar that updates after every command. It shows the current state — sent, acknowledged, or failed — with appropriate color coding. This gives the operator immediate feedback without needing to check the packet log.

Telemetry Display

Telemetry data is shown in two separate panels. This separation matches the physical structure of a CanSat — the outer container and the inner payload are two distinct units that send different data.

6.1 Container Telemetry

Field	Unit	Normal Range	Description
Packet Number	—	0 to 9999+	Sequential counter, padded to 4 digits
Altitude	m	0 to 1000+	Height above ground, derived from pressure

6.

Pressure	hPa	900 to 1013	Atmospheric pressure, decreases with altitude
Temperature	°C	-40 to +40	Ambient air temperature
Descent Rate	m/s	8 to 10	Speed of descent during parachute phase
Battery Voltage	V	3.8 to 4.2	Main battery, turns red below 3.8V
Voltage	V	~3.7 to 4.1	System operating voltage (98% of battery)

6.2 Payload Telemetry

Field	Unit	Description
Roll	degrees	Rotation around the longitudinal axis of the CanSat
Pitch	degrees	Nose-up or nose-down tilt angle
Yaw	degrees	Compass heading, rotates 0 to 360
GPS Latitude	decimal deg.	North-South position coordinate
GPS Longitude	decimal deg.	East-West position coordinate
GPS Satellites	count	Number of satellites currently in view
GPS Fix	—	NO FIX / FIX 2D / FIX 3D — color coded green/orange/red

6.3 Update Rate and Color Coding

Both panels update every second when telemetry is running. Individual values are color-coded dynamically — battery voltage turns red when low, GPS fix turns green for 3D fix and red for no fix, and descent rate text changes color based on whether the mission is in ascent or descent phase.

Error Code System

error code system is a 4-digit binary display in the lower part of the left column. It gives the operator an instant visual summary of mission health without needing to read individual telemetry numbers. Each digit is either 0 (normal) or 1 (fault).

7.1 Digit Definitions

Digit	Condition Monitored	Value 0 (Green)	Value 1 (Red / Fault)
D1	Descent Rate	Rate is within 8 to 10 m/s safe range	Rate is outside safe range

7.

The

D2	GPS Availability	GPS satellites in view (4 or more)	GPS satellites fewer than 4
D3	Payload Separation	Separation completed successfully	Separation failure detected
D4	Emergency Parachute	Parachute is inactive (normal)	Emergency parachute deployed

7.2 Error Code Examples

Code	Meaning
0000	All systems normal — no faults detected
1000	Descent rate fault — falling too fast or too slow
0100	GPS data unavailable — satellite lock lost
0010	Payload separation failure
0001	Emergency parachute has been deployed
1111	All four fault conditions are active simultaneously

7.3 Visual Design of Error Indicators

Each digit is displayed as a large square box (36x36 pixels). In normal state, the box has a green border, green text, and a soft green background glow. In fault state, the box turns red and a CSS keyframe animation pulses the opacity between full and half, making the fault extremely visible even from a distance. Alongside the digit boxes, four colored dot indicators with text labels provide additional redundancy so faults are never missed.

Real-Time Graphs

Five real-time graphs are displayed in a 2-column grid in the center of the dashboard. They are built using Chart.js version 4.4.1 and update every second as new telemetry packets arrive.

8.1 Graph Details

Graph	Color	Y-Axis Unit	Width
Altitude	Cyan (#00d4ff)	meters	Half width
Pressure	Purple (#a78bfa)	hPa	Half width
Temperature	Orange (#f97316)	°C	Half width

8.

Descent Rate	Red (#ff3b3b)	m/s	Half width
Battery Voltage	Green (#00ff88)	Volts	Full width (spans both columns)

8.2 Sliding Window Buffer

Each graph holds a maximum of 120 data points in its data array. When a new point arrives and the array is already full, the oldest point is removed from the front using `Array.shift()`. This creates a sliding window that always shows the most recent 120 seconds of data. Memory usage stays constant regardless of how long the mission runs.

8.3 Performance Optimizations

- Chart animation duration is set to 0 so updates are instant with no rendering lag
- Point radius is set to 0 so no dots are drawn, keeping the line clean and fast
- Y-axis tick count is limited to 4 maximum to avoid cluttered labels
- X-axis is hidden because the sliding window makes timestamps redundant

The battery voltage graph spans the full width at the bottom of the chart grid because it is the most mission-critical power indicator and deserves more horizontal space to show trends clearly.

GPS Tracking Map

GPS tracking map shows the real-time position of the CanSat on an interactive map. It uses Leaflet.js as the mapping framework and OpenStreetMap as the tile provider. Both are completely free and open-source.

9.1 Map Features

- A circular marker with a cyan glow shows the current GPS position of the CanSat
- A blue polyline traces every position point since the mission started, creating the full trajectory history
- GPS coordinates are displayed in decimal degrees in the panel header, updated with each packet
- The map auto-centers on the first few GPS positions but then stays fixed so the operator can zoom and pan freely without the view jumping
- Standard Leaflet zoom controls allow the operator to zoom in for detail or zoom out for context

9.

The

9.2 Simulation Behavior

During simulation mode, the GPS position starts near New Delhi (close to the India Space Lab address) and drifts by a small random amount each second. This simulates realistic GPS movement during descent. The drift values are small enough that the trajectory stays within a few hundred meters, similar to a real CanSat mission in a small launch area.

10. Orientation Visualization

Two different orientation displays are provided side by side in the right column. Both respond to the same Roll, Pitch, and Yaw telemetry values.

10.1 Three.js 3D CanSat Model

A 3D model of the CanSat is rendered in real time using WebGL through the Three.js library. The model is built from basic Three.js geometry primitives:

- CylinderGeometry for the main body of the can
- ConeGeometry for the pointed nose cone at the top
- BoxGeometry for four rectangular fins at the base, spaced 90 degrees apart

Directional lighting comes from the upper right to give the model depth and shadow. Ambient lighting fills in the shadow side so the model does not go completely dark. An AxesHelper is added to the scene to show the X, Y, Z orientation axes as colored lines.

The model rotation is updated every animation frame using linear interpolation (lerp) at 10% per frame toward the target Roll, Pitch, and Yaw values. This means the model never snaps to a new angle suddenly — it always eases smoothly into position, which looks much more realistic than direct value assignment.

10.2 Artificial Horizon

The artificial horizon is drawn on an HTML5 Canvas element using only the 2D Canvas API — no external library is needed. It mimics the Attitude Indicator instrument found in aircraft cockpits.

The sky half uses a blue gradient from dark navy to medium blue. The ground half uses a brown gradient from dark soil to lighter earth tone. The horizon line between them is white. Pitch ladders (horizontal lines with degree labels) are drawn at every 10 degree interval. A fixed yellow crosshair in the center represents the CanSat's reference attitude. A small yellow circle on the roll arc at the outer edge shows the current bank angle.

The canvas context is saved and restored around the rotated drawing section so the outer frame and crosshair remain fixed while only the sky-ground portion rotates with roll and shifts with pitch.

11.

The

Live Video Streaming

video streaming section uses the browser's built-in MediaDevices API. No external plugin, extension, or library is required. It works in any modern browser that supports getUserMedia.

11.1 Implementation Details

- On page load, the browser queries available media devices using `navigator.mediaDevices.enumerateDevices()`
- All video input devices are listed in the camera selection dropdown
- When the operator clicks START CAM, the browser shows a permission dialog asking to allow camera access
- If permission is granted, the camera stream is assigned to an HTML video element and plays automatically
- A red blinking REC indicator appears as an overlay on the video when streaming is active
- Clicking STOP CAM calls `track.stop()` on all camera tracks, releasing the hardware

In a real CanSat mission, this section would receive video from a wireless camera transmitter aboard the CanSat. In the current implementation it uses the ground PC's webcam to demonstrate the complete interface. The mechanism for displaying the stream is identical in both cases.

12. Data Management Features

12.1 Telemetry Logging

Every telemetry packet received during a session is pushed into a JavaScript array called `telemetryLog`. This array grows throughout the mission and holds all received data until the page is closed or Reset Packets is clicked. Each entry in the array is a JavaScript object with all 14 telemetry fields.

12.2 CSV Export

Clicking the Export CSV button converts the `telemetryLog` array into a properly formatted CSV string. The first row is a header line with all 14 field names. Each subsequent row is one packet. The CSV is wrapped in a Blob object with MIME type `text/csv` and downloaded automatically using a temporary anchor element. The filename includes the Unix timestamp to prevent accidental overwriting.

The 14 CSV fields are: packet, altitude, pressure, temperature, descent_rate, battery, voltage, roll, pitch, yaw, lat, lon, sats, fix

12.3 Graph Export

The altitude chart canvas is exported as a PNG image using the `canvas.toDataURL('image/png')` method. This method returns a base64-encoded image string which is set as the href of a temporary anchor element and downloaded. The current implementation exports the altitude graph. All charts use the same canvas-based approach, so any of the five charts could be exported the same way.

12.4 Packet Log

A scrolling text log at the bottom of the center column records every event — packets received, commands sent, commands acknowledged, camera events, and system messages. Log entries are color-coded: white for normal, orange for warnings, and red for errors. The log holds up to 200 lines before auto-trimming from the top. It auto-scrolls to the latest entry. A CLEAR button wipes the log without affecting the telemetry data.

12.5 Reset Function

The Reset Packets button clears the `telemetryLog` array, resets the packet counter and mission statistics to zero, calls `chart.data.labels = []` and `chart.data.datasets[0].data = []` on all five charts, clears the GPS path array and removes the polyline from the map, and resets the max altitude and min temperature statistics. The display goes back to its initial state as if the page was just loaded.

13.

The

Testing Strategy

assignment specifies using the WeGyanik Kit microcontroller as a real-time telemetry streaming device. The GCS supports both a built-in simulator for immediate testing and a real hardware connection via Web Serial API for Arduino integration.

13.1 Built-in Flight Simulator

The simulator generates realistic telemetry across four flight phases:

Phase	Trigger Time	Key Behavior
Pre-Launch	0 to 5 seconds	Baseline values, no GPS satellites, descent rate zero
Ascent	5 to 40 seconds	Altitude rises 8-12 m/s, pressure drops, temp decreases
Descent	40 to 120 seconds	Altitude falls at 8-10 m/s, descent rate enters fault zone near end
Landed	After 120 seconds	Altitude near zero, all motion stops, GPS stabilizes

Roll and Pitch oscillate using sine and cosine functions to simulate realistic wobble during descent. Yaw increases linearly to simulate slow spin. Battery voltage decreases slowly over time. GPS satellites gradually appear during ascent phase.

13.2 Arduino Uno Integration

For testing with the WeGyanik Kit Arduino Uno, the following steps apply:

1. Download and install Arduino IDE from <https://www.arduino.cc/en/software> (free)
2. Program the Arduino Uno to send comma-separated telemetry packets over USB serial at 9600 baud
3. Open cansat_gcs.html in Google Chrome
4. Add a Connect Serial button that calls `navigator.serial.requestPort()` — Chrome will show a port selection dialog
5. Read incoming serial data, split by comma, and feed values into the `applyTelemetry()` function

The packet format the Arduino should send:

```
PKT_NUM,ALTITUDE,PRESSURE,TEMP,DESCENT,BATT,VOLT,ROLL,PITCH,YAW,LAT,LON,SATS,FIX
```

13.3 Test Cases Verified

6. All 14 telemetry fields update correctly every second
7. All five charts update smoothly with no visible lag or stutter
8. Error code D1 activates correctly when simulated descent rate goes outside 8-10 m/s
9. Error code D2 activates correctly when GPS satellites drop below 4
10. Emergency Parachute command requires confirmation before D4 changes to 1
11. CSV export downloads correctly and opens in Excel with all columns properly separated
12. Graph export downloads as a valid PNG image file
13. Camera starts and stops correctly with permission dialog appearing in Chrome
14. Reset Packets clears all data including charts, map, and statistics
15. Mission Elapsed Timer counts up correctly from zero

14. Results and Observations

The GCS was completed and tested successfully. The table below shows how the project performed against the assignment evaluation criteria:

Evaluation Criteria	Weight	Implementation	Status
UI/UX Design	15%	Dark aerospace theme, 3-column grid, consistent typography	Complete
Telemetry Handling	20%	14 fields, 1Hz update rate, container + payload separation	Complete
Real-Time Visualization	20%	5 Chart.js graphs, 120-point sliding window, instant update	Complete
Mission Control Features	15%	5 commands, safety dialogs, live status feedback	Complete
Graphing and Tracking	10%	Leaflet map, live marker, full trajectory polyline	Complete
Orientation and Video Systems	10%	Three.js 3D model, artificial horizon, webcam stream	Complete
Code Quality and Scalability	10%	Single file, modular functions, clean separation of concerns	Complete

One key observation was that smooth animation in the 3D model required linear interpolation rather than direct value assignment. Direct assignment caused the model to jerk between positions on each packet, which looked unrealistic. Using lerp at 10% per frame made the motion look natural and continuous even at only 1 packet per second.

Another observation was that Chart.js performs much better with animation disabled for realtime applications. With the default animation settings, the chart would animate every update, causing a visible lag as each new data point was drawn. Setting animation duration to 0 made the charts update instantly.

15. Challenges Faced

- Fitting all features into a single HTML file without losing readability. The solution was to organize JavaScript into clearly named function groups with comment headers separating each section, similar to how modules would work in a larger project.
- The artificial horizon required careful canvas state management. The roll rotation and pitch translation had to be applied inside a save()/restore() block so the outer frame and crosshair stayed fixed while only the inner sky-ground area moved. Getting the clipping mask and rotation center right took several iterations.
- Leaflet.js uses light-colored tiles by default which looked wrong against the dark dashboard. The fix was to use standard OpenStreetMap tiles and add a CSS dark

overlay to the map container, or simply accept the standard tile look since the marker and polyline are clearly visible regardless.

- The Three.js r128 version does not include OrbitControls in the main bundle, so the 3D model could not be made interactive (drag to rotate). The model is instead animated automatically based on telemetry, which is more appropriate for a mission display anyway.
- Camera access from a local HTML file (file:// protocol) is blocked in some Chrome configurations. The solution is to either serve the file through a local web server or change Chrome flags. For submission, this is noted in the documentation.

16. Conclusion

This project produced a fully functional, single-page Ground Control Software for a CanSat mission. Every section of the India Space Lab assignment brief was implemented and tested. The software runs in Google Chrome with no installation required, making it easy to demonstrate and share.

The project demonstrated that modern web technologies are capable enough to build genuine mission operations software. Features like WebGL (via Three.js), real-time canvas drawing, interactive maps, live camera access, and file exports were all achieved within a single browser tab using standard web APIs.

Working on this project gave a much deeper understanding of how real aerospace GCS systems are designed — the emphasis on immediate readability, the need for redundant fault indicators, the importance of operator safety through confirmation dialogs, and the value of organized data logging. These design patterns come from decades of human factors research in aviation and space programs and apply equally at the student CanSat level.

If this project were extended further, the most impactful next steps would be connecting it to a real Arduino Uno via Web Serial API to receive live sensor data, and adding a WebSocket server so multiple ground operators could monitor the same mission simultaneously from different computers.

17. References

16. India Space Lab — CanSat and CubeSat Satellite Project Assignment Brief, 2026
 17. Chart.js Official Documentation — <https://www.chartjs.org/docs/latest/>
 18. Leaflet.js Official Documentation — <https://leafletjs.com/reference.html>
 19. Three.js Official Documentation — <https://threejs.org/docs/>
 20. OpenStreetMap — <https://www.openstreetmap.org/>
 21. MDN Web Docs — Web Serial API — https://developer.mozilla.org/enUS/docs/Web/API/Web_Serial_API
 22. MDN Web Docs — MediaDevices.getUserMedia — <https://developer.mozilla.org/enUS/docs/Web/API/MediaDevices/getUserMedia>
 23. MDN Web Docs — CSS Grid Layout — https://developer.mozilla.org/enUS/docs/Web/CSS/CSS_grid_layout
 24. MDN Web Docs — WebSocket API — <https://developer.mozilla.org/enUS/docs/Web/API/WebSocket>
 25. MDN Web Docs — Canvas API — https://developer.mozilla.org/enUS/docs/Web/API/Canvas_API
 26. MDN Web Docs — Blob API — <https://developer.mozilla.org/enUS/docs/Web/API/Blob>
 27. Arduino Official Documentation — <https://docs.arduino.cc/>
-

— *End of Report* —

India Space Lab | CanSat & CubeSat Satellite Project | June 2026